

Auditoria Técnica e Metodológica — Mercado Albion (Américas)

Data: 11/06/2026 · **Escopo:** todo o código da plataforma (backend albion/ + app.py, frontend web/, CLI `analyze.py`, testes, documentação) e a metodologia estatística/econométrica embutida.

Método: leitura integral dos arquivos, verificação das fórmulas contra os fatos validados do jogo/API (wiki oficial + Albion Online Data Project, jun/2026), execução da suíte de testes e inspeção do cache real.

Estado geral: a plataforma está funcional e bem encaminhada — taxas corretas, motor de flips correto (incl. a regra de qualidade do Mercado Negro), cache disciplinado, 7/7 testes passando. Os problemas abaixo são o que separa um bom scanner de uma central de inteligência confiável.

Severidade: **[ALTA]** Alta (corretude/decisão financeira) · **[MÉDIA]** Média (qualidade/robustez) · **[BAIXA]** Baixa (polimento).

1. Achados de corretude e robustez

[ALTA] A1. O `opportunity_score` é relativo ao próprio conjunto de resultados

`app.py` -> `_score_recommendations()`: lucro, potencial/dia e liquidez são normalizados pelo **máximo da consulta atual**. Consequências: o mesmo flip pode ser "executar" hoje e "cautela" amanhã sem que nada tenha mudado nele — basta os vizinhos da lista mudarem; com 1 resultado, ele sempre ganha 100 nos três fatores; scores não são comparáveis entre consultas, dias ou filtros — exatamente o que um sistema de recomendação para guild não pode ser.

Correção: normalizar contra escalas absolutas e estáveis (ex.: lucro em escala log com âncoras fixas; liquidez em faixas absolutas de itens/dia; ROI já é absoluto) ou contra percentis históricos da própria plataforma (`price_snapshots`). Documentar a régua no glossário.

[ALTA] A2. Item Lab é cache-only sem nenhum caminho de coleta na própria aba

`web/app.js` -> `runHist()` chama `/api/history` e `/api/item-analysis` sempre com `cache_only=true`, e não existe botão "buscar dados novos". O histórico local cobre **68 de 12.066 itens** (0,6%): o laboratório estatístico enxerga quase nada do mercado, e o usuário não tem como alimentá-lo a não ser rodando scans com volume em outra aba.

Correção: (a) botão "Coletar da API" no Item Lab (com aviso de rate limit); (b) watchlist persistida + coletor agendado (o docs/OBJETIVOS já prevê `watchlist_items/collection_runs` — é a pendência mais valiosa da plataforma).

[ALTA] A3. SQLite sem WAL nem `busy_timeout` com dois processos escrevendo

`albion/client.py` -> `AODP.__init__`: o lock (`threading.Lock`) só protege threads do **mesmo** processo. O fluxo real do usuário é servidor aberto + `analyze.py` no terminal: dois processos gravando em `data/cache.db` em modo journal padrão -> erros `database is locked` intermitentes (escritas perdidas/comandos abortados).

Correção: ao conectar, `PRAGMA journal_mode=WAL; PRAGMA busy_timeout=5000;` (e `synchronous=NORMAL` para desempenho). Três linhas, elimina a classe inteira de falha.

[ALTA] A4. `print()` no tratamento de erro do proxy de ícones pode derrubar a requisição

`app.py` -> `_download_icon()`, ramo `except`: `print(f"[ícone] falha no download: {e}")` — mensagens de erro do Windows em PT-BR têm acentos; em console cp1252 isso lança `UnicodeEncodeError` **dentro do handler de exceção** e a requisição vira 500. É o mesmo bug que já aconteceu nesta base (o ramo `rc != 0` foi corrigido para logar em arquivo; este ramo ficou para trás).

Correção: logar no mesmo `icon_errors.log` em bytes, como o outro ramo.

[MÉDIA] A5. Sem retry para erros 5xx da API

`albion/client.py` -> `_get()`: `raise_for_status()` está fora do laço de retry — um 502/503 esporádico da AODP (comum em serviço comunitário) aborta o scan inteiro na hora, enquanto erros de transporte e 429 têm retry.

Correção: tratar `r.status_code >= 500` como retryável com backoff.

[MÉDIA] A6. `daily_potential` confunde "sem dado" com "volume zero"

`app.py` (scan) e `analyze.py` (CLI): `if vb and vs` — um volume diário 0 (dado real: ninguém vendeu) é tratado como ausência de dado (None/-). Um item ilíquido aparece igual a um item não medido. **Correção:** `if vb is not None and vs is not None`, exibindo 0 como 0 (que mata a oportunidade, como deve).

[MÉDIA] A7. Crescimento sem política de retenção

`price_snapshots` é append-only sem limite (`cache.db`: 18 MB no primeiro dia) e dobra cada escrita de preços; `data/icons/` e `fetch_log` também só crescem. **Correção:** retenção configurável (ex.: snapshots brutos 90 dias + agregados diários permanentes), VACUUM periódico, e tabelas derivadas diárias (o guia já as recomenda) para 30/90/180 dias rápidos.

[MÉDIA] A8. Cache de histórico re-busca quando só a janela muda

`client.py` -> `get_history()`: a chave do `fetch_log` inclui `days`; pedir 7 dias depois de já ter 30 frescos no cache dispara nova ida à API. Custo de rede desnecessário. **Correção:** marcar frescor por (item, cidade, escala) com a maior janela coberta.

[MÉDIA] A9. `get_gold` não tem TTL

Toda visita à página + intervalo de 5 min batem na API (a tabela `gold` é gravada mas nunca lida). Inconsistente com o resto do design de cache.

[BAIXA] A10. Outros pontos de robustez

- `datetime.utcnow()` deprecado (`client.py`) — trocar por `datetime.now(timezone.utc)`.
- Favicon aponta para o render externo (pode falhar como os ícones; usar `/icon/T4_BAG`).
- Chips de qualidade/encanto: desmarcar tudo equivale a "todas" silenciosamente.
- Limite silencioso de 500 flips ocultos no `localStorage`.
- `_cached_price_rows` lê a tabela inteira e filtra em Python — ok hoje; com snapshots/preços na casa dos 10^5 registros, mover filtros de metadados para SQL (tabela `items` espelhada no SQLite).
- `app.py` virou monólito de ~950 linhas misturando API, estatística e proxy — extrair `albion/stats.py` (também destrava testes unitários da camada estatística).

2. Achados metodológicos (estatística/econometria)

O Item Lab é um bom esqueleto (VWAP, dispersão, momentum, qualidade do dado, interpretação prudente), mas várias escolhas comprometem a validade inferencial:

[ALTA] B1. Z-score sobre níveis de preço de séries não-estacionárias

`app.py` -> `_analyze_history_series()`: o `z` usa média e desvio dos **níveis** na janela. Preços de itens em tendência (pós-patch, mudança de meta) não são estacionários: numa queda estrutural, o preço atual SEMPRE terá `z` negativo, e a stance "investigar compra" recomenda segurar a faca caindo. É o erro clássico de aplicar estatística i.i.d. a série temporal.

Correção acadêmica: (a) calcular o `z` sobre os **resíduos da tendência** (já existe a regressão linear — use-a) ou sobre **retornos**; (b) alternativa robusta: distância à mediana em unidades de MAD; (c) reportar junto a inclinação para o leitor distinguir "barato" de "despencando".

[ALTA] B2. Intervalo da previsão usa o desvio errado

`naive_forecast_next = último + slope ± 1·desvio dos níveis`. O desvio dos níveis numa série em tendência mede majoritariamente a própria tendência, não a incerteza da previsão -> intervalos largos demais em tendência e estreitos demais em reversão; a cobertura nominal (~68%?) nunca é declarada.

Correção: usar o desvio dos **resíduos** da regressão (ou o erro-padrão de previsão completo, que inclui a incerteza dos coeficientes), declarar a cobertura, e validar empiricamente: % de vezes que o realizado caiu no intervalo (backtest de calibração). Melhor ainda: intervalos por quantis empíricos dos resíduos.

[ALTA] B3. Volume da AODP é censurado — e o "Pot./dia" ignora isso

item_count só é registrado quando algum jogador com o cliente de coleta abre aquele mercado: é um **limite inferior** do volume real, com censura não-aleatória (mercados populares são mais observados). $Pot./dia = lucro \times \min(vol_origem, vol_destino)$ ainda assume que você captura 100% do volume diário ao spread atual, sem competição nem slippage.

Correção: tratar volume como piso censurado (documentar); aplicar um fator de captura conservador (ex.: 10–25%, calibrável por backtest); modelar slippage quando houver livro de ordens (NATS).

[MÉDIA] B4. Janelas em pontos, não em tempo

MA "rápida/lenta" = últimas 7/30 **observações** e momentum = 7 pontos: em escala horária isso é 7h/30h, em diária 7d/30d — mesmos rótulos na UI para coisas diferentes. Padronizar janelas em horas/dias e rotular dinamicamente.

[MÉDIA] B5. Mediana e outliers

median_proxy não interpola para n par (viés sistemático para cima em amostras pequenas); o filtro de outliers das recomendações é uma régua fixa $0,35 \times -3 \times VWAP$ — arbitrária, derruba oportunidades legítimas em repricing de patch e deixa passar manipulação dentro da faixa. Migrar ambos para mediana real + bandas por MAD/quantis por item.

[MÉDIA] B6. confidence ignora liquidez

A confiança do flip usa apenas o frescor das duas pontas: um preço de 29 min num item que vende 1 unidade/semana ganha "alta". Incorporar liquidez/frequência (o worklog já lista como pendência) — e, idealmente, a **probabilidade de sobrevivência da ordem** estimada dos snapshots (análise de sobrevivência simples: quantos % das ordens observadas a idade X ainda existem em X+delta — os price_snapshots já permitem começar isso).

[MÉDIA] B7. Sem sazonalidade nem regime

O mercado do Albion tem ciclo semanal forte (fim de semana) e choques de patch/evento. Nenhuma métrica condiciona ao dia-da-semana, e a comparação "preço vs média da janela" mistura regimes. Primeiro passo barato: médias por dia-da-semana no Item Lab e flag de "janela contém patch" (calendário manual de patches).

[BAIXA] B8. Validação inexistente da camada estatística

Não há nenhum teste numérico de VWAP, z-score, slope, percentil ou score (os testes da API são só de contrato), nem backtest de calibração das previsões/stances. Antes de qualquer modelo mais sofisticado, criar: testes unitários com séries sintéticas de resposta conhecida + um backtest contínuo que registre acerto das stances ("investigar compra" rendeu o quê em 7 dias?).

3. Produto, CLI e documentação

- [MÉDIA] C1. A camada nova (recomendações, descoberta, item-analysis) **não existe na CLI** — analyze.py é a interface das análises automatizadas (inclusive do Claude) e ficou uma geração atrás da UI. Adicionar analyze.py recommend e analyze.py lab <item>.
- [MÉDIA] C2. Exports inconsistentes: CSV do scanner usa ; + decimal vírgula + BOM (Excel BR); CSV da CLI usa , + decimal ponto. Unificar (parâmetro --csv-dialect ou padrão único BR).
- [MÉDIA] C3. Documentação defasada: CLAUDE.md/AGENTS.md descrevem 5 abas e 8 comandos (hoje são 6 abas e 12 endpoints); README não menciona Início/Item Lab/score. Para um time de guild usar, o README precisa de uma seção "como interpretar o score e a confiança".
- [MÉDIA] C4. Lixo na raiz: sw.json é uma página de erro 404 em HTML salva com nome .json (apagar), _tmp_scale_bench.py, _tmp_status_bench.py, server_stdout.log/server_stderr.log vazios. Mover benchmarks para scripts/ ou remover.
- [BAIXA] C5. Testes dependem do cache real (data/cache.db, items_db.json) — passam/falham conforme o estado da máquina. Isolar com fixtures temporárias.
- [BAIXA] C6. search da CLI não aceita --ench; UI permite 180 dias × escala horária (a AODP retém pouco histórico horário — pedido grande para resposta vazia).
- [BAIXA] C7. Glossário: entrada "Conf." correta e honesta — manter esse padrão de honestidade quando A1/B1 forem corrigidos (descrever a régua do score).

4. O que já está certo (não mexer)

- Fórmulas de taxas exatamente corretas (4%/8% + 2,5% nas duas pontas de ordem; compra instantânea sem taxa) e cobertas por teste.
 - Regra de qualidade do Mercado Negro (item q preenche ordem $\leq$ q) correta, inclusive com o fix do filtro de qualidade; BM restrito a equipamento de combate e somente venda.
 - Throttle de janela dupla respeita os dois limites oficiais; chunking respeita o limite de URL; linhas `city="0"` descartadas; datas placeholder viram None.
 - Separação prices (estado atual) $\times$ price_snapshots (histórico) — arquitetura certa para backtesting.
 - Honestidade da UI: idades coloridas por ponta, aviso de flip fantasma, cobertura do cache exibida, previsão rotulada como baseline.
-

5. Roadmap recomendado (ordem de execução)

Fase 0 — Fundação de dados (1 sessão): WAL + busy_timeout (A3); fix do print (A4); retry 5xx (A5); retenção/agregados diários (A7); limpeza da raiz (C4).

Fase 1 — Coleta disciplinada: watchlist persistida + coletor agendado com collection_runs auditável (A2); botão de coleta no Item Lab; gold com TTL (A9).

Fase 2 — Estatística robusta: z de resíduos/MAD (B1); intervalos calibrados (B2); janelas em tempo (B4); mediana real (B5); score absoluto (A1); confiança com liquidez (B6); testes numéricos (B8).

Fase 3 — Microestrutura: análise de sobrevivência de ordens nos snapshots (probabilidade de flip fantasma por idade); haircut de captura no Pot./dia (B3); sazonalidade semanal (B7).

Fase 4 — Backtesting: simulador walk-forward com taxas, re-listagens (taxa de anúncio em cada edição!), fill probability e slippage; métricas: PnL, hit rate, drawdown máximo, exposição. Toda regra (score, stance) só vira recomendação oficial da guild depois de sobreviver ao backtest.

Fase 5 — Predição probabilística: baselines (passeio aleatório, média móvel) como piso obrigatório; depois ETS/ARIMA sazonal ou quantile regression por item-cidade com amostra mínima; sempre com intervalo e cobertura validada out-of-sample.

Fase 6 — Camada de guild: relatórios exportáveis, alocação de capital entre rotas (Kelly fracionário com teto), alertas.

6. Limitações desta auditoria

web/style.css foi inspecionado superficialmente (cosmético); scripts/build_items_db.py e run.bat foram auditados em sessão anterior e não mudaram; nenhum teste de carga foi executado; o comportamento do coletor NATS/dumps da AODP (fase de livro de ordens) não foi avaliado por ainda não existir no projeto. Nenhum arquivo do projeto foi alterado por esta auditoria além da criação de AUDITORIA.md e AUDITORIA.pdf.